Connessione al Database - Assistenza AI

Di seguito i prompt utilizzati per risolvere problematiche relative alla connessione al database SQL Server nel progetto BikeStores.

Prompt 1: Stringa di connessione con Windows Authentication

Richiesta:

```
Mi serve la stringa di connessione per SQL Server con Windows Authentication.
Devo connettermi al database BikeStores su localhost.
```

Problema:
La soluzione doveva usare autenticazione integrata di Windows (nessun login SQL), senza dover configurare utenti e password nel file di configurazione.

Soluzione adottata:

```
{
  "ConnectionStrings": {
    "BikeStoreConnection":
"Server=localhost;Database=BikeStores;Trusted_Connection=True;TrustServerCertificate=True;"
  }
}
```

`Trusted_Connection=True` abilita l'autenticazione Windows. `TrustServerCertificate=True` necessario in sviluppo locale per bypassare la validazione del certificato SSL del server locale.

Prompt 2: DbContext unico condiviso tra i progetti

Richiesta:

```
Ho tre progetti (Backend MVC, WebAPI, Frontend MVC) che devono
accedere allo stesso database BikeStores. Come organizzo il DbContext?
Devo creare un contesto per ogni progetto?
```

Problema:
Rischiavo di duplicare la configurazione del DbContext in ogni progetto, creando disallineamenti nelle query e difficoltà di manutenzione.

Soluzione adottata:
DbContext centralizzato in `BikeStores.Data`, progetto libreria di classi condiviso:

```
// BikeStores.Data/Models/BikeStoresContext.cs
public class BikeStoresContext : DbContext
{
    public BikeStoresContext(DbContextOptions<BikeStoresContext> options)
        : base(options) { }

    public DbSet<Brand> Brands => Set<Brand>();
    public DbSet<Category> Categories => Set<Category>();
    public DbSet<Product> Products => Set<Product>();
    // ... resto DbSet
}
```

Registrato in ogni progetto via `Program.cs`:

```
builder.Services.AddDbContext<BikeStoresContext>(options =>
    options.UseSqlServer(connectionString));
```

Prompt 3: Dual DbContext per Identity (solo Backend)

Richiesta:

```
Il Backend MVC deve usare Identity per autenticazione, ma i dati
di business (prodotti, clienti, ordini) stanno nel database BikeStores
già esistente. Come separo le tabelle Identity senza toccare il database
di business?
```

Problema:
Identity genera tabelle `AspNetUsers`, `AspNetRoles`, etc. che non devono essere mescolate con le tabelle `production.products`, `sales.customers` del database BikeStores. Ma non potevo creare un secondo database - il DBA non lo permette.

Soluzione adottata:
Dual DbContext sullo stesso database, con Identity che usa il proprio schema:

```
// BikeStores.Backend/Data/ApplicationDbContext.cs
public class ApplicationDbContext : IdentityDbContext<IdentityUser>
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options) { }

    protected override void OnModelCreating(ModelBuilder builder)
    {
        base.OnModelCreating(builder);
        builder.HasDefaultSchema("identity");
    }
}
```

```
// Program.cs
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(connectionString));

builder.Services.AddDbContext<BikeStoresContext>(options =>
    options.UseSqlServer(connectionString));

builder.Services.AddIdentity<IdentityUser, IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>();
```

Le migrazioni Identity vanno in `Data/Migrations/`, separate dal modello di business. Il database non viene mai alterato per le tabelle di business.

Prompt 4: AsNoTracking() per query read-only nelle API

Richiesta:

```
La WebAPI restituisce prodotti in sola lettura. Siccome non ci sono
operazioni di scrittura, come evito che Entity Framework tenga traccia
di tutti gli oggetti caricati?
```

Problema:
Ogni GET /api/products caricava centinaia di entità nel change tracker di EF, consumando memoria inutilmente. Le API non salvano mai dati, solo lettura.

Soluzione adottata:
`AsNoTracking()` su tutte le query read-only:

```
public async Task<ActionResult<IEnumerable<Product>>> GetProducts(
    [FromQuery] short? modelYear,
    [FromQuery] int? brandId,
    [FromQuery] int? categoryId,
    [FromQuery] string? search)
{
    var query = _context.Products
        .Include(p => p.Brand)
        .Include(p => p.Category)
        .AsNoTracking()               // <-- nessun tracking
        .AsQueryable();

    if (modelYear.HasValue)
        query = query.Where(p => p.ModelYear == modelYear.Value);

    return await query
        .OrderBy(p => p.ProductName)
        .ToListAsync();
}
```

Pattern applicato anche a BrandsController, CategoriesController, StoresController.

Prompt 5: Connessione Multi-Progetto dallo stesso Solution

Richiesta:

```
La stringa di connessione è uguale per i tre progetti della soluzione.
Come evito di duplicarla in ogni appsettings.json?
```

Problema:
Tre progetti (Backend, WebAPI, Frontend) hanno lo stesso identico `ConnectionStrings:BikeStoreConnection` in tre file diversi. Se cambia il server, devo aggiornare tutti.

Soluzione adottata:
Anche se duplicata, ogni progetto ha la propria copia in `appsettings.json` per permettere esecuzione indipendente. Per evitare inconsistenze, documentato in questo file e sincronizzato manualmente:

```
Server=localhost;Database=BikeStores;Trusted_Connection=True;TrustServerCertificate=True;
```

Tutti e tre i progetti leggono la stessa chiave:

```
var connectionString = builder.Configuration
    .GetConnectionString("BikeStoreConnection");
```

Riepilogo

Problema	Soluzione
Autenticazione Windows	`Trusted_Connection=True`
DbContext condiviso	Libreria `BikeStores.Data`
Identity separato	Dual DbContext, schema `identity`
Performance lettura	`AsNoTracking()`
Multi-progetto	Stessa connessione, stessi settings